Advanced Data Management
Quiz Refresher: Chapter 5, Answer Sheet

Answers to the 8 quick checks, the 3 problem situations, the real plan and the discussion tickets, in the rubric's shape: letter or number first, formula, computation, one-sentence decision, trap

Advanced Data Management

2026-10-29

Part A: Quick checks

A1. **(b) 50.** Equality on a column with no MCV list: selectivity 1/n_distinct = 1/10,000; 500,000 × 1/10,000 = 50. (d) is the number of distinct values, not the matches.

A2. **(b) 20 %.** Buckets of width 10, each holding 10 %: 15–25 contributes half (0.05), 25–35 whole (0.10), 35–45 half (0.05): 0.20, so 100,000 rows. (a) counts only the whole bucket; (b) is also what uniformity gives, 20/100, which is the point of an equi-depth histogram on uniform data.

A3. **(b) 8,185.** $3{,}185 \times 1.0 + 500{,}000 \times 0.01 = 3{,}185 + 5{,}000$. (c) adds the filter term $500{,}000 \times 0.0025 = 1{,}250$ that only a `WHERE` brings; (a) forgets the per-row CPU; (d) is $3{,}185 \times 4$, random prices for a sequential scan.

A4. **(b) 1,000.** $\max(\text{est}/\text{act}, \text{act}/\text{est}) = 20{,}000/20$. An *under*-estimate by 1,000, the dangerous direction: a believed 20-row input invites a nested loop or a tiny hash build. (a) is the ratio the wrong way round; q-error is never below 1.

A5. **(b) too low.** Independence multiplies $0.40 \times 1/\text{n_distinct(city)}$; since city implies country, the truth is just the city count, larger than the product. Correlation in an AND makes the product under-estimate (the mistake to avoid is answering "too high").

A6. **(b) 120 and 31.** Left-deep orders $n! = 5! = 120$; DP keeps the best plan per non-empty subset, $2^5 - 1 = 31$, built level by level. (c) confuses subsets with orders; $n!$ is not the number of plans either (algorithms and access paths multiply it).

A7. **(b).** Selection pushdown is an algebraic identity, $\sigma_p(R \bowtie S) = \sigma_p(R) \bowtie S$ when p touches only R: always answer-preserving, needs no statistics. (a), (c), (d) are physical choices made by costing.

A8. **(b) stale statistics, `ANALYZE orders`.** The catalog still holds the counts of the last `ANALYZE`; a year of inserts tripled the table and nothing

refreshed `reltuples`. Stale is failure 1; correlation and skew give wrong *fractions*, not a wrong base count.

Part B: Problem situations

Solution: B1 "Just add an index on amount"

(a) `amount > 95`: the bucket 95–105 lies entirely above 95, the one bucket entirely above contributes 1/10, no partial bucket. Selectivity $s = 0.10$, rows $0.10 \times 500{,}000 = 50{,}000$. Packed together they would fill $50{,}000/157 \approx 318$ pages.
(b) Seq Scan: $3{,}185 \times 1 + 500{,}000 \times 0.01 + 500{,}000 \times 0.0025 = 3{,}185 + 5{,}000 + 1{,}250 = 9{,}435$.
Index Scan: descent $3 \times 4 = 12$, heap $50{,}000 \times 4 = 200{,}000$, CPU $50{,}000 \times 0.01 = 500$: about 200,512, twenty-one times the scan. The index is read in `amount` order but the rows sit wherever they were inserted, so every match is a random page.
(c) $4sN < B \Leftrightarrow s < B/(4N) = 3{,}185/2{,}000{,}000 = 0.16\,\%$, about 800 rows. On a uniform 5–105 column that is the top 0.16 of the range: roughly `amount > 104.84`. The report keeps 10 %, sixty times that threshold.
Decision: the index is useless for this report (and costs every insert a second write); the planner is right. The correlation of 0.002 means matching rows are spread over essentially all 3,185 pages, so even a bitmap heap scan, which reads each page once in order, would touch about every page and gain nothing over the scan. With correlation near 1 (table physically ordered by amount) the 50,000 rows would sit in 318 consecutive pages and an index or BRIN scan would win by a factor of ten.

Solution: B2 The Casablanca report is slow

(a) Country from the MCV list: 0.40. City, no MCV list: 1/n_distinct = $1/200 = 0.005$. Independence: $0.40 \times 0.005 \times 10{,}000 = 20$ customers. Truth: 3,000. q-error = $3{,}000/20 = 150$, an under-estimate. Failure 2, correlated columns: city determines country, so the AND is really just the city predicate, and even that alone is wrong (3,000 against $0.005 \times 10{,}000 = 50$), which is failure 3, skew: Casablanca is a most-common value that the 200-value column did not list.
(b) Join on a key: $|R||S|/\max(d_R, d_S)$ with $d = 10{,}000$ on both sides. Estimated $20 \times 500{,}000/10{,}000 = 1{,}000$ rows; true $3{,}000 \times 50 = 150{,}000$. The q-error is still 150: the leaf error is carried up unchanged, and every operator above (the aggregate, any further join) is sized for 1,000 rows instead of 150,000.
(c) Index nested loop: $B_R + |R| \times$ (index pages + matching heap pages). Believed: $1 + 20 \times (3 + 50) = 1{,}061$. Hash join: build on the filtered customers (1 page), $B_R + B_S = 1 + 3{,}185 = 3{,}186$. The planner picks the **index nested loop**, 3 times cheaper on paper. Real: $1 + 3{,}000 \times 53 = 159{,}001$ random page reads against the hash join's 3,186 sequential

ones, which barely change (3,000 customers still fit in one or two pages of the build). The chosen plan is about 50 times more expensive than the alternative, in random I/O at that: on HDD, 159,000 × 10 ms is 27 minutes; the hash join streams 25 MB.

(d) No: `ANALYZE` refreshes counts and per-column statistics, and both columns' statistics are already correct one at a time; the error is the *product*. Fix:

```
CREATE STATISTICS s_geo (dependencies) ON country, city FROM
    customers;
ANALYZE customers;
```

With the dependency city → country known, the planner uses the city selectivity alone, 50 rows under uniformity; raising the statistics target so that Casablanca enters the MCV list brings it to about 3,000 (then 3,000 × 53 = 159,001 > 3,186 and the hash join is chosen).

Solution: B3 Three tables, which join first?

(a) Filtered customers 0.40 × 10,000 = 4,000; filtered products 10; orders 500,000 (no predicate of its own).

(b) Products with orders: $10 \times 500{,}000 / \max(10, 200)$: careful, after the filter the products side has 10 distinct ids but orders still has 200, so divide by 200: 25,000 (one twentieth of the orders, as expected for 10 of 200 products).

Customers with orders: 4,000 × 500,000/10,000 = 200,000.

Products with customers: no join predicate, a cross product, 10 × 4,000 = 40,000 rows that every order would then have to be matched against.

Final result either way: 25,000 × 4,000/10,000 = 10,000, and 200,000 × 10/200 = 10,000. Same answer from two orders, as it must be: the order changes the cost, never the result.

(c) Products first: its intermediate is 25,000 rows against 200,000. The second join then processes eight times fewer rows, and if it is a hash join the 10 products are a one-page build. System-R DP defers cross products: a pair with no join predicate is not considered while a connected pair exists, so the third order is never costed.

(d) Joins commute and associate, so the written order is only a suggestion; the planner reorders all tables inside one `FROM` list up to `join_collapse_limit` (default 8, and 3 tables are far below it). With `join_collapse_limit = 1` the explicit `JOIN` syntax would be executed in the written order, customers with orders first, 200,000 rows into the second join.

Part C: Read a real plan

Solution: Ticket REP-31, questions 1 to 6

1. The tree. Limit ← Sort ← HashAggregate ← Hash Join with two children: the Index Scan on orders (probe side) and the Hash node over the Seq Scan on products (build side).

Node	All input first?	Why
Seq Scan products	no	streams pages
Hash	**yes**	the table must be complete before any probe
Index Scan orders	no	emits each matching row as it is found
Hash Join	no (probe side)	streams the probe once the build is done
HashAggregate	**yes**	a group's sum is unknown until the last input row
Sort	**yes**	even a top-N heapsort must see every row to know the top 5
Limit	no	stops after 5 rows

2. Where `rows=4` comes from. Index Cond `cust_id = 42`: no MCV list, so $1/10{,}000 \times 500{,}000 = 50$ rows. Filter `order_date >= 2025-12-01`: histogram fraction $43{,}388/500{,}000 = 0.0868$. Independence multiplies them: $50 \times 0.0868 = 4.3$, printed as 4. The assumption is that which customer ordered and when are unrelated.

3. q-error and failure. $45/4 = 11.25$ (10.4 against the unrounded 4.3). Failure 2, correlation: for this customer the two predicates are not independent; 45 of their 52 orders are in December (`rows=45` kept plus 7 removed). `ANALYZE` changes nothing, the per-column statistics are already right. `CREATE STATISTICS (dependencies)` will not help either: a dependency statistic measures how much `cust_id` determines `order_date` across the *whole* table, and across 10,000 customers it does not; this is one customer's pattern. Some estimates are simply beyond the catalog.

4. `Buffers: shared hit=55`. 3 index pages (root, internal, leaf) plus one heap page per order fetched: $45 + 7 = 52$ orders, so $3 + 52 = 55$. One page per order because the correlation of `cust_id` is 0.001: a customer's orders were inserted over the year among everyone else's, so consecutive index entries point to unrelated pages. All hits, none read: the pages were in shared buffers from earlier runs.

5. Index on `(cust_id, order_date)`. The date moves from `Filter` into `Index Cond`, the line `Rows Removed by Filter: 7` disappears, and the 7 rejected rows are never fetched from the heap: `Buffers` drops to about $3 + 45 = 48$. The estimate stays `rows=4`: estimation uses statistics, not indexes; an index changes the cost of a path, not the predicted rows. An index on `order_date` alone would be worse: it selects 43,388 rows, 8.7 % of the table, and the planner would answer with a bitmap or sequential scan over thousands of pages instead of 52 fetches. Lead with the selective column.

6. Why harmless, and when not. Both possible plans are cheap: the build side is a 2-page table of 200 products, and whether the probe has 4

or 45 rows changes nothing; a nested loop against 2 pages would also be fine. The factor 11 is a disaster when it sizes a decision: a believed 4-row input chosen as the outer of an index nested loop against a big table, or as a hash build that is actually 11 times larger than `work_mem` (Grace with many batches), or as the left side of a further join whose estimate is then also 11 times too small, and so on up the tree.

Part D: Discussion

Solution: Ticket D1 "Set random_page_cost = 1 everywhere"

Claim. Lowering `random_page_cost` is reasonable on SSD, but 1.0 overstates the case and flips plans that should not flip; 1.1 to 1.5 is the usual compromise, and it should be measured, not decreed.

Mechanism. The cost model charges 1.0 for a page read in a sequential scan and `random_page_cost` for a page fetched through an index. The ratio is what decides the access path: the index wins when $s \cdot N \cdot \text{random_page_cost} < B$, that is below $s < B/(4N) = 1/(4 \times 157) \approx 0.16\,\%$ on `orders` today, before bitmap scans soften it. With `random_page_cost = 1` the threshold becomes $s < B/N = 1/157 \approx 0.64\,\%$, four times higher (about 3,200 rows instead of 800), and index nested loops become four times cheaper on paper relative to hash and merge joins.

What flips. Queries keeping between 0.16 % and 0.64 % of a table move from Seq or Bitmap Scan to Index Scan; joins with a small-looking outer move from hash to index nested loop. On SSD a random read is still not free: it is one 8 KB request at about 100 µs against 16 KB per 100 µs streamed at 500 MB/s, and a sequential scan also benefits from read-ahead and from the ring buffer. More importantly, pages already in shared buffers cost the same either way, so the real ratio depends on the hit rate, which is why the cost is a model and not a measurement.

What can go wrong. Every under-estimated cardinality is now amplified: a nested loop chosen on a believed 20 rows that are really 3,000 (B2) is 150 times more I/O, and the lower random cost makes the planner reach for that loop more often. Servers that still have HDD tiers or a cold cache after restart will see index scans that are slower than the scans they replaced.

Measure. Before: collect the slow-query log and the plans of the top queries with `EXPLAIN (ANALYZE, BUFFERS)`, and the cache hit rate from `pg_statio_user_tables`. Change the parameter per session first (`SET random_page_cost = 1.1`), rerun the same queries, compare actual times, not costs (costs are in arbitrary units and never comparable across settings). Only then set it per tablespace or server, and keep 4.0 where disks are spinning.

Solution: Ticket D2 "We need optimizer hints"

Claim. A 40-minute nested loop is almost always a cardinality error, not a planner bug; fix the input to the planner before overriding its output.
Mechanism. The planner chose the loop because it believed the outer side was small, a few thousand rows at most, so $|R| \times$ (index pages + matches) looked cheaper than reading the inner table once. The believed count was wrong by orders of magnitude, from one of three causes: stale statistics (the table grew since the last `ANALYZE`), correlated predicates multiplied as if independent, or a skewed value missing from the MCV list. Costing itself is exact arithmetic; the row count fed into it was the lie, and the lie is multiplied at every join above the leaf.
Order of fixes. (1) Statistics: `ANALYZE` the tables; if a multi-column filter is still off, `CREATE STATISTICS (dependencies)` on those columns and analyse again; if a frequent value is mis-estimated, raise `default_statistics_target` (or per column) so that it enters the MCV list. (2) Cost parameters: `random_page_cost` on SSD, `effective_cache_size` to reflect the real cache. (3) Memory: `work_mem` so that the right hash join or sort does not spill. (4) Experiments: `SET enable_nestloop = off` *in one session* to see what the planner's second choice costs, then `RESET`; never as a server setting. (5) Only then `pg_hint_plan`, for a query whose data the statistics cannot describe (a per-customer pattern like REP-31).
Why hints rot. A hint freezes one plan for one data distribution. Next year the table is ten times larger, the hinted nested loop that was right at 2,000 rows is wrong at 20,000, and nobody remembers the hint exists; the optimizer, given correct statistics, would have changed its mind on its own. That is why PostgreSQL has no hints in the core: the levers it offers all improve what the planner knows, so every query benefits, not only the one that hurt today.

Summary: If you missed these, reread chapter 5

- A1, A2, A5, B1(a), B2(a), C2: Sections 3.2 (histograms) and 4.1 (the selectivity table), P1.
- A3, B1(b)–(c), D1: Sections 5.1–5.2 (cost model, access path), P7.
- A6, A7, B3: Sections 2.2 (pushdown), 6.1–6.2 (search space, System-R DP), P5, P6.
- A4, A8, B2(b)–(d), C3, C6, D2: Sections 4.3 (propagation), 7.1–7.3 (the three failures), 7.4 (reading EXPLAIN), P3, P4.
- C1, C4, C5: Section 7.4 plus chapter 4 (pipeline breakers), chapter 3 (composite index), chapter 6 (Buffers).